

Mini project guidelines (Python)

Introduction

Mini projects help students apply programming concepts to real-world problems. These projects are designed to improve logical thinking, coding skills, and problem-solving ability using Python.

Each project integrates multiple concepts such as functions, loops, conditionals, and file handling, making learning practical and meaningful.

Objectives of Mini Projects

The main objectives are:

- To apply **Python programming concepts in real-life scenarios**
- To improve **problem-solving and analytical skills**
- To understand **data storage and file handling**
- To develop **modular programming using functions**
- To gain experience in building **small applications**

General Guidelines

Students must follow these guidelines:

- ✓ The project should be **simple and easy to understand**
- ✓ Use a **menu-driven approach** wherever possible
- ✓ Use **functions** for modular programming
- ✓ Include **input validation**
- ✓ Use **file handling (Text/CSV/JSON)**
- ✓ Add **comments** for readability
- ✓ Handle errors using **exception handling**

General Guidelines

Students must follow these guidelines:

- ✓ The project should be **simple and easy to understand**
- ✓ Use a **menu-driven approach** wherever possible
- ✓ Use **functions** for modular programming
- ✓ Include **input validation**
- ✓ Use **file handling (Text/CSV/JSON)**
- ✓ Add **comments** for readability
- ✓ Handle errors using **exception handling**

RECOMMENDED MINI PROJECTS (WITH FUNCTION NAMES)

1. Student Record Management System

Objective

The objective of this project is to help students understand how to **store, process, and manage structured data** using Python.

Suggested Function Names

- add_student()
- calculate_total_average()
- determine_result()
- save_to_file()
- read_from_file()
- display_students()

Description

A program to manage student details such as name, marks, and results.

Features

- Add student details
- Calculate total and average
- Determine pass/fail
- Save data in file
- Display stored records

Concepts Covered

- Data types and variables
- Functions
- Conditionals
- File handling
- Exception handling

2. Simple Banking System

Objective

This project simulates real-world banking operations and helps students understand **transaction processing and validations**.

Suggested Function Names

- create_account()
- deposit_amount()
- withdraw_amount()
- check_balance()
- save_account_data()
- load_account_data()

Description

A basic banking application to perform deposit, withdrawal, and balance checking.

Features

- Create account
- Deposit money
- Withdraw money
- Display balance
- Store data in file

Concepts Covered

- Functions
- Loops
- Conditionals
- File handling
- Exception handling

3. To-Do List Application

Objective

This project helps students build a simple **task management system**.

Suggested Function Names

- add_task()
- view_tasks()
- delete_task()
- save_tasks()
- load_tasks()

Description

A program to manage daily tasks.

Features

- Add task
- View tasks
- Delete task
- Store tasks in file

Concepts Covered

- Lists
- File handling
- Loops
- Functions

4. Quiz Application

Objective

The objective is to create an interactive system using **decision-making and data structures**.

Suggested Function Names

- load_questions()
- display_question()
- get_user_answer()

- `check_answer()`
- `calculate_score()`
- `show_result()`

Description

A program that asks questions and calculates score.

Features

- Display questions
- Accept answers
- Calculate score
- Show result
- Load questions (optional JSON)

Concepts Covered

- Dictionaries
- Loops
- Conditionals
- Functions
- JSON handling

5. Number Guessing Game

Objective

This project introduces **game logic and user interaction**.

Suggested Function Names

- `generate_number()`
- `get_user_guess()`
- `check_guess()`
- `display_hint()`
- `count_attempts()`

Description

User guesses a randomly generated number.

Features

- Random number generation
- Hint system
- Count attempts

Concepts Covered

- Loops
- Conditionals
- Random module

6. CSV Employee Data Analyzer

Objective

This project teaches students how to **analyze real-world data** stored in CSV files.

Suggested Function Names

- read_csv_file()
- display_records()
- filter_employees()
- find_highest_salary()
- sort_records()

Description

Read and analyze employee data from CSV file.

Features

- Display records
- Filter employees
- Find highest salary
- Sort data

Concepts Covered

- CSV handling
- Lists
- Loops
- Conditionals

7. Contact Management System

Objective

This project helps students manage data using **dictionaries and JSON**.

Suggested Function Names

- add_contact()
- view_contacts()
- search_contact()
- save_contacts()
- load_contacts()

Description

A system to store and manage contacts.

Features

- Add contact
- View contacts
- Search contact
- Save in JSON

Concepts Covered

- Dictionaries
- JSON handling
- File handling

- Functions

8. Simple Calculator

Objective

This project strengthens understanding of **basic operations and control flow**.

Suggested Function Names

- add()
- subtract()
- multiply()
- divide()
- display_menu()

Description

A menu-driven calculator program.

Features

- Arithmetic operations
- Continuous execution
- Error handling

Concepts Covered

- Functions
- Loops
- Operators
- Exception handling